Grouping Related Callbacks Together

If you have a group of related callbacks, it may be convenient to group them into a separate handler class. These handlers can be shared between multiple models. A handler class is simply a class that defines callback methods (`before_save`, `after_create`, and so on). Create the source files for these handler classes in `app/models`.

In the model object that uses the handler, you create an instance of this handler class and pass that instance to the various callback declarations. A couple of examples will make this clearer.

If our application uses credit cards in multiple places, we might want to share our `normalize_credit_card_number` method across multiple models. To do that, we'd extract the method into its own class and name it after the event we want it to handle. This method will receive a single parameter, the model object that generated the callback:

```
class CreditCardCallbacks

  # Normalize the credit card number
  def before_validation(model)
    model.cc_number.gsub!(/[-\s]/, '')
  end
end
```

Now, in our model classes, we can arrange for this shared callback to be invoked:

```
class Order < ApplicationRecord
  before_validation CreditCardCallbacks.new
  # ...
end

class Subscription < ApplicationRecord
  before_validation CreditCardCallbacks.new
  # ...
end
```

In this example, the handler class assumes that the credit card number is held in